

DATE & TIME FUNCTIONS

✓ Most used functions

SELECT NOW(); -- current date + time

SELECT CURDATE(); -- only date

SELECT CURTIME(); -- only time

◆ Extract parts from date

SELECT YEAR('2026-05-05'); -- 2026

SELECT MONTH('2026-05-05'); -- 5

SELECT DAY('2026-05-05'); -- 5

👉 Real table:

SELECT YEAR(hire_date), MONTH(hire_date)

FROM employees;

◆ Date arithmetic

SELECT DATE_ADD('2026-05-05', INTERVAL 10 DAY);

SELECT DATE_SUB('2026-05-05', INTERVAL 1 MONTH);

◆ Difference between dates

SELECT DATEDIFF('2026-05-10', '2026-05-01'); -- 9 days

💡 Real-world use cases

- Partition data by **year/month**
 - Calculate **user retention**
 - Find **tenure of employees**
 - Time-based filtering in pipelines
-

⚠ Interview traps

- DATEDIFF ignores time (only date)
- Timezone issues (very common in production)

🔥 2. INT (NUMERIC) FUNCTIONS

◆ Basic ones

```
SELECT ABS(-10);    -- 10
SELECT CEIL(4.2);   -- 5
SELECT FLOOR(4.9);  -- 4
SELECT ROUND(4.567,2); -- 4.57
SELECT TRUNCATE(4.567,2); -- 4.56
```

◆ Random

```
SELECT RAND(); -- 0 to 1
SELECT FLOOR(1 + RAND()*10); -- 1 to 10
```

💡 Real-world use

- Data cleaning
 - Bucketing values
 - Sampling data
 - Financial calculations
-

⚠️ Interview trap

👉 ROUND vs TRUNCATE (you already saw this — very common question)

🔥 3. CAST (Type Conversion)

✅ Syntax

```
CAST(expression AS datatype)
```

◆ Examples

```
SELECT CAST('123' AS SIGNED); -- string → int
SELECT CAST(123 AS CHAR);    -- int → string
SELECT CAST('2026-05-05' AS DATE);
```

◆ **With your case (VARCHAR date)**

```
SELECT YEAR(CAST(hire_date AS DATE))  
  
FROM employees;
```

💡 **Real-world use**

- Cleaning messy data
 - Schema transformation in ETL
 - Joining different data types
-

⚠️ **Interview traps**

- Implicit vs explicit conversion
👉 Always prefer **CAST** (explicit = safer)
-

🔥 **4. CASE WHEN (Conditional Logic)**

✅ **Syntax**

```
CASE  
  
  WHEN condition THEN value  
  
  WHEN condition THEN value  
  
  ELSE value  
  
END
```

◆ **Example**

```
SELECT name,  
  
CASE  
  
  WHEN salary > 100000 THEN 'High'  
  
  WHEN salary > 50000 THEN 'Medium'  
  
  ELSE 'Low'  
  
END AS salary_category  
  
FROM employees;
```

◆ Simple CASE

CASE department

WHEN 'IT' THEN 'Tech'

WHEN 'HR' THEN 'Support'

END

💡 Real-world use

- Data transformation
 - Categorization (very common)
 - Feature engineering (ML pipelines)
-

⚠️ Interview traps

- Missing ELSE → returns NULL
 - Order matters (top-down evaluation)
-

🔥 5. COALESCE (NULL Handling)

✅ Concept

👉 Returns **first non-null value**

COALESCE(val1, val2, val3)

◆ Examples

SELECT COALESCE(NULL, NULL, 10); -- 10

◆ Practical

SELECT COALESCE(salary, 0) FROM employees;

◆ Multiple fallback

SELECT COALESCE(phone, email, 'No Contact')

FROM users;

Real-world use

- Handling missing data
 - Default values in reports
 - Avoid NULL in aggregations
-

Interview traps

- Stops at first non-null (does NOT check further)
 - Data types must be compatible
-

FINAL INTERVIEW SUMMARY (VERY IMPORTANT)

Topic	Key Idea
Date funcs	Used in partitioning & time analysis
Int funcs	Rounding vs truncation matters
CAST	Always prefer explicit conversion
CASE	SQL's IF-ELSE logic
COALESCE	NULL handling (very common in ETL)

Real Data Engineering Scenario

SELECT

user_id,

YEAR(CAST(event_time AS DATE)) AS year,

CASE

WHEN amount > 1000 THEN 'High Value'

ELSE 'Normal'

END AS category,

COALESCE(amount, 0) AS safe_amount

FROM transactions;

 This single query uses:

- CAST
- YEAR

- CASE
- COALESCE

✓ This is exactly how real pipelines look.

Question → Answer → Key Insight (interview point)

MYSQL INTERVIEW NOTES (Q&A FORMAT)

1. Extract Year & Month from VARCHAR Date

Question

hire_date is stored as VARCHAR. Extract **year and month safely**.

Answer

SELECT

```
YEAR(STR_TO_DATE(hire_date, '%Y-%m-%d')) AS year,  
MONTH(STR_TO_DATE(hire_date, '%Y-%m-%d')) AS month
```

FROM employees_new;

Key Insight

- Always use STR_TO_DATE for **safe conversion**
 - Avoid implicit conversion
-

2. Calculate Employee Tenure

Question

Find how many **years each employee has worked**.

Answer

SELECT

```
employee_id,  
  
TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) AS tenure  
FROM employees;
```

 Key Insight

- TIMESTAMPDIFF is more accurate than dividing days
-

 3. Salary Categorization

 Question

Categorize salary into High / Medium / Low.

 Answer

```
SELECT  
  
    name,  
  
    CASE  
  
        WHEN salary > 100000 THEN 'High'  
  
        WHEN salary >= 50000 THEN 'Medium'  
  
        ELSE 'Low'  
  
    END AS category  
FROM employees;
```

 Key Insight

- CASE runs **top-down**
 - Order matters
-

 4. Replace NULL Values

 Question

Replace:

- NULL amount → 0
- NULL status → 'PENDING'

 Answer

```
SELECT  
  
    COALESCE(amount, 0) AS amount,
```

COALESCE(status, 'PENDING') AS status

FROM orders;

 Key Insight

- COALESCE returns first non-null value
-

 5. Date Difference with NULL Handling

 Question

Find days between order_date and delivery_date.
If delivery_date is NULL → use current date.

 Answer

```
SELECT
    DATEDIFF(
        COALESCE(delivery_date, CURDATE()),
        order_date
    ) AS days_diff
FROM orders;
```

 Key Insight

- DATEDIFF ignores time part
-

 6. Efficient Random Sampling

 Question

Fetch 5 random rows from a large table efficiently.

 Answer

```
SELECT *
FROM employees
WHERE id >= (
    SELECT FLOOR(RAND() * (SELECT MAX(id) FROM employees))
)
LIMIT 5;
```

 Key Insight

- Avoid ORDER BY RAND() → very slow

🔥 7. TRUNCATE vs ROUND

? Question

Show difference and explain usage.

✅ Answer

SELECT

TRUNCATE(99.999, 2) AS truncated, -- 99.99

ROUND(99.999, 2) AS rounded; -- 100.00

💡 Key Insight

- TRUNCATE → cuts
- ROUND → adjusts
- Financial systems usually prefer **ROUND**

🔥 8. Join with Different Data Types

? Question

Join:

- users.user_id (INT)
- orders.user_id (VARCHAR)

✅ Answer

SELECT *

FROM users u

JOIN orders o

ON u.user_id = CAST(o.user_id AS UNSIGNED);

💡 Key Insight

- Avoid mismatched joins → bad performance
- Fix schema if possible

🔥 9. Monthly Revenue Report

? Question

Get year, month, total revenue (ignore NULLs).

✅ **Answer**

```
SELECT  
  
    YEAR(transaction_date) AS year,  
  
    MONTH(transaction_date) AS month,  
  
    SUM(COALESCE(amount, 0)) AS revenue  
  
FROM transactions  
  
GROUP BY year, month;
```

💡 **Key Insight**

- COALESCE avoids NULL issues in aggregation
-

🔥 **10. Real ETL Scenario (Combined Concepts)**

❓ **Question**

From events table:

- Convert VARCHAR → DATE
- Extract year
- Replace NULL revenue
- Categorize revenue

✅ **Answer**

```
SELECT  
  
    YEAR(STR_TO_DATE(event_time, '%Y-%m-%d')) AS year,  
  
    COALESCE(revenue, 0) AS revenue,  
  
    CASE  
  
        WHEN COALESCE(revenue, 0) > 1000 THEN 'High'  
  
        ELSE 'Normal'  
  
    END AS category  
  
FROM events;
```

💡 **Key Insight**

👉 **Combines:**

- Date conversion
- NULL handling

- Conditional logic

✓ This is how **real pipelines** look

BONUS QUESTION (IMPORTANT)

Question

SELECT COALESCE(NULL, '10', 20);

Answer

 '10'

Key Insight

- First non-null value is returned
 - Result type becomes **STRING**
-

FINAL REVISION CHEAT SHEET

- STR_TO_DATE → safe conversion
 - TIMESTAMPDIFF → accurate date difference
 - CASE → SQL IF-ELSE
 - COALESCE → NULL handling
 - CAST → explicit conversion
 - Avoid ORDER BY RAND()
-